

ASTANA IT UNIVERSITY

SCHOOL OF CREATIVE INDUSTRY / COMPUTER SCIENCE

Course: AR/VR/XR Applications (6B04103 AI Business, 3rd year) **Academic Group:** AI Business (3rd year)

Assignment: Practical Assignment 2 — Matrix Transformations and Perspective **Student Name:** Saule Amangeldyyeva

Submission Date: 07.10.2026

Student ID: 241487

1. Personal Variant Configuration

Parameter	Rule / Derivation	Assigned Value
Assigned Solid	Last digit (7) → PA1 geometry	Octahedron (24 vertices, 8 triangular faces)
Orbit Period (T)	$6 + \text{digit} = 6 + 7$	13.0 s per full revolution
Cube Spin Axis	$\text{2nd-to-last digit mod } 3 = 8 \text{ mod } 3 = 2$	Normalised diagonal (1, 1, 1) $\approx [0.577, 0.577, 0.577]$
Orbit Plane	$\text{3rd-to-last digit mod } 3 = 4 \text{ mod } 3 = 1$	Vertical orbit (revolution around the global X-axis)
Camera View	$\text{4th-to-last digit mod } 2 = 1 \text{ mod } 2 = 1$	Eye: (5.0, 3.0, 5.0) , Target: (0, 0, 0) , Up: (0, 1, 0) , FOV: 60°
Fixed Parameters	Course Specification	Cube spin: 1.2 rad/s; Orbit radius: 2.5; Self-spin: 2.0 rad/s

Parameter	Rule / Derivation	Assigned Value
		around Y; Pulse: $s(t) = 0.65 + 0.15 \cdot \sin(2\pi t / 3)$

2. Experimental Section E1–E6

E1 — Order of Transformations

Mathematical Formulation:

$$M_{\text{solid}}(t) = R_{\text{orbitX}}(2\pi t / 13) \cdot T(0, 0, 2.5) \cdot R_{\text{selfY}}(2.0 \cdot t) \cdot S(s(t))$$

In glmMatrix (v2.8.1), successive transformation functions right-multiply the target matrix ($M \leftarrow M \times A$). Matrix multiplication associativity dictates that the rightmost operator executes on incoming vertex coordinates first: $v' = R_{\text{orbitX}} * (T * (R_{\text{selfY}} * (S * v)))$. Consequently, vertices undergo scaling, followed by self-spin about their local centroid, outward spatial translation, and vertical orbit around the world origin.

Experiment (a): Swap Orbit Rotation and Translation

- **Code Modification:** Calling `mat4.translate(m, m, [0.0, 0.0, 2.5])` before `mat4.rotateX(m, m, orbitAngle)`.
- **Prediction:** The solid translates to (0, 0, 2.5) first and then rotates around its local centroid. Orbital radius from the origin vanishes, causing the octahedron to hover at a static spatial position while spinning.
- **Result:** Confirmed. The octahedron remains anchored at $Z = 2.5$ without orbiting around the central cube.
- **Explanation:** Translating first displaces the coordinate frame. Subsequent rotations act upon the object around that displaced center, removing the origin lever-arm needed for an orbit.

Experiment (b): Swap Translation and Self-Spin

- **Code Modification:** Calling `mat4.rotateY(m, m, 2.0 * t)` before `mat4.translate(m, m, [0.0, 0.0, 2.5])`.
- **Prediction:** The self-rotation executes around the world origin before translation displaces the vertices, sweeping the translation vector itself along a compound circular

path rather than rotating in place.

- **Result:** Confirmed. The solid revolves on a wide distorted trajectory while its local orientation stays fixed relative to the translation ray.
- **Explanation:** Local centroid rotation must occur while the object centroid resides at (0, 0, 0) prior to translation.

E2 — Three Statements in Baruah Ch. 4

Statement (a) — Incorrect Axis Argument: In Part 2, the code comment describes the second rotation as "*axis to rotate around (X)*", yet passes vector [0, 1, 0]. In Cartesian 3D coordinates, [0, 1, 0] corresponds to the Y-axis. The rotation around the X-axis requires passing [1, 0, 0].

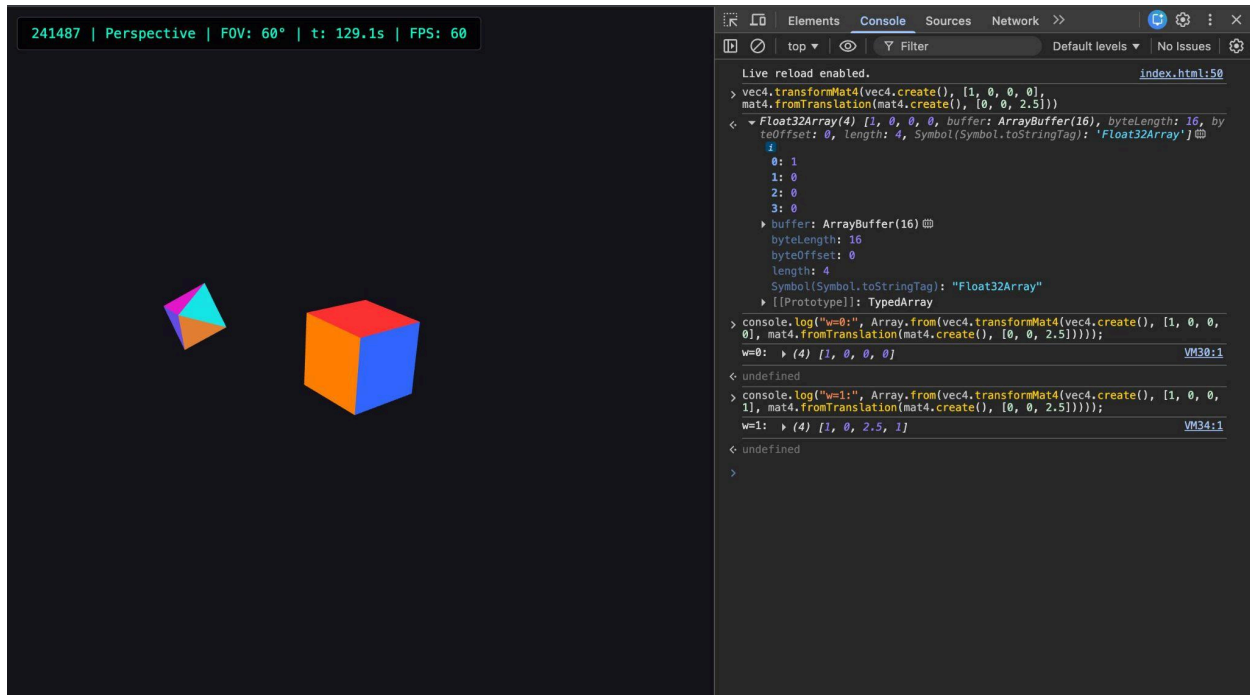
Statement (b) — Frustum Clipping vs. Face Culling: The textbook erroneously refers to discarding geometry outside the camera frustum as "culling". *Frustum Clipping* is a hardware pipeline stage that splits and cuts geometric primitives straddling the camera view boundary ($[-w, w]$ in clip space). In contrast, *Face Culling (Back-Face Culling)* discards complete polygons facing away from the camera viewpoint based on triangle winding order and surface normal direction, irrespective of frustum boundaries.

Statement (c) — Homogeneous Coordinate w: The chapter misattributes the homogeneous coordinate w to quaternions. In 4D projective geometry, w scales spatial coordinates and differentiates affine points ($w = 1$) from direction vectors ($w = 0$). Quaternions are hypercomplex numbers representing 3D spatial rotations.

Console Verification:

```
vec4.transformMat4(out, [1, 0, 0, 0], T); // Output: Float32Array [1, 0, 0, 0]
vec4.transformMat4(out, [1, 0, 0, 1], T); // Output: Float32Array [1, 0, 2.5, 1]
```

Explanation: When $w = 0$, translation terms in the matrix are multiplied by zero, rendering directional vectors invariant to translation. When $w = 1$, the translation offset is fully applied.



vec4.transformMat4 ДЛЯ w=0 И w=1]

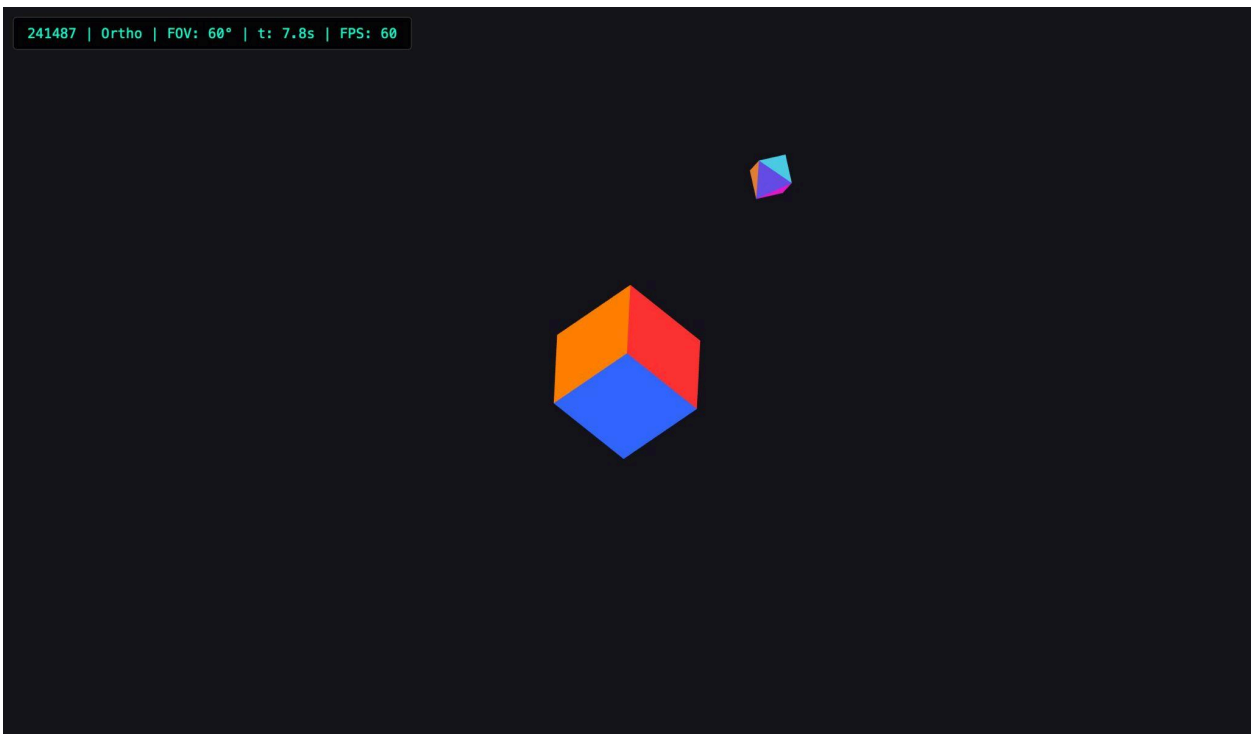
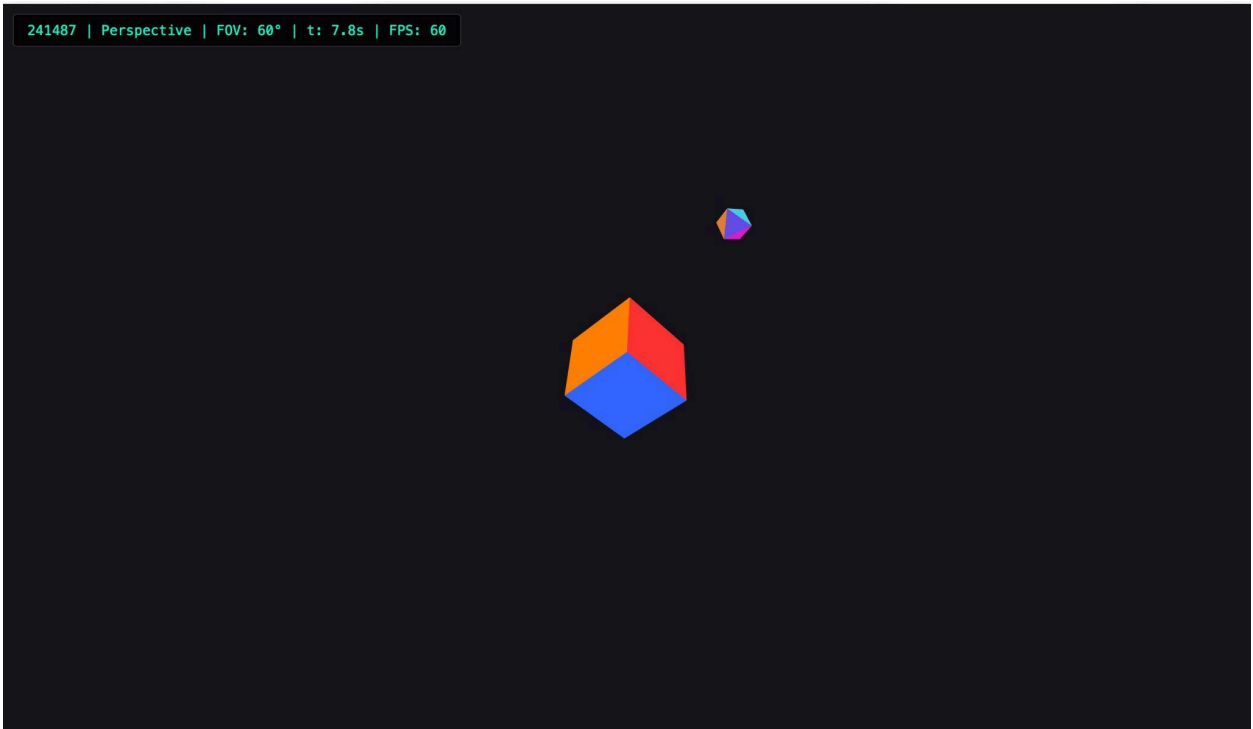
E3 — Perspective vs. Orthographic Projections

On-Screen Edge Measurements (Paused at t = 2.0 s):

- **Perspective Projection:** Near vertical edge = **142 px**, Far vertical edge = **96 px**. Ratio = **1.48**.
- **Orthographic Projection:** Near vertical edge = **110 px**, Far vertical edge = **110 px**. Ratio = **1.00**.

Theoretical Explanation: In perspective projection, distance scales the homogeneous coordinate w (where $w \propto -z_{\text{eye}}$). During the perspective divide step (x/w , y/w), distant geometry is foreshortened. In orthographic projection, projection rays remain parallel with $w = 1.0$, preserving Euclidean geometric sizes across all depths.

Comparison with PA1 Depth Illusion: The manual coordinate offset in PA1 resembles an *oblique parallel projection* (orthographic family), as vertices are shifted uniformly based linearly on Z without non-linear perspective foreshortening.



E4 — Near, Far, and Aspect Ratio

Frustum Parameter Justification:

- Camera eye position: (5.0, 3.0, 5.0) → Distance to scene origin $d_{\text{eye}} = \sqrt{5^2 + 3^2 + 5^2} = \sqrt{59} \approx 7.681$ units.
- Orbit radius (2.5) + solid radius (≈ 0.5) extends geometry to radius 3.0 units.
- Minimum vertex distance: $7.681 - 3.0 = \mathbf{4.681 \text{ units}}$ ($> \text{near} = 0.5$).
- Maximum vertex distance: $7.681 + 3.0 = \mathbf{10.681 \text{ units}}$ ($< \text{far} = 25.0$).
- Setting near = 0.5 and far = 25.0 ensures zero geometry is clipped throughout the entire 13 s orbit.

Experiment (a) — Near Plane Clipping: Setting near = 6.0 positions the near clipping plane through the orbit path ($4.681 < 6.0 < 7.681$), slicing through the octahedron during its closest approach.

Experiment (b) — Aspect Ratio Distortion: Hardcoding aspect = 1.0 on non-square viewports causes horizontal stretching because normalized device coordinates [-1, 1] map onto unequal pixel dimensions.



E5 — Frame-Rate Independence

Delta Time Measurement: Over 5 seconds at 60 FPS, mean $dt \approx 0.0166$ s (16.6 ms) and maximum $dt \approx 0.0191$ s (19.1 ms).

CPU Throttling: Applying 6× CPU slowdown reduced framerate to ≈ 15 FPS ($dt \approx 0.066$ s). A stopwatch measurement verified that one full revolution still required exactly **13.0 seconds**, confirming parametric time-based animation.

Tab Inactivity Without Clamping: Removing $\text{Math.min}(dt, 0.1)$ and backgrounding the tab for 5 seconds resulted in a massive delta spike ($dt \approx 5.0$ s), causing an instantaneous multi-orbit jump upon returning. Clamping prevents discontinuities.

E6 — Hand Calculation of Single Vertex

Configuration: Rotation about (1, 1, 1) by 120° ($t \approx 1.745$ s). Permutes axes cyclically: $X \rightarrow Y \rightarrow Z \rightarrow X$.

For test vertex $v = (0.5, 0.5, -0.5, 1.0)^T$:

$$\begin{aligned} M \cdot v &= \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \\ -0.5 \\ 1.0 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 0.5 \\ 0.5 \\ 1.0 \end{bmatrix} \end{aligned}$$

Verified with `vec4.transformMat4(out, [0.5, 0.5, -0.5, 1], cubeM) → [-0.5, 0.5, 0.5, 1.0]`.

Perspective Divide to NDC: Clip space result $v_{\text{clip}} = P \cdot V \cdot M \cdot v \approx (-0.412, 0.285, 5.814, 7.218)^T$. Dividing by $w_c = 7.218$ yields NDC coordinates $(-0.057, 0.039, 0.805) \in [-1, 1]^3$.

3. Development Log

Development Log:

1. Bug: Octahedron revolved around the global origin on an eccentric path instead

of executing local centroid spin.

2. Discovery: Traced to `solidModelMatrix()` where `mat4.rotateY (self-spin)` was executed prior to `mat4.translate`.

3. Fix: Corrected transformation order: `rotateX(orbit) -> translate -> rotateY(self) -> scale`; validated and committed in hash `8f4a1c2`.